
Playwright para .NET 10

Pruebas de navegador fiables para Razor Pages, Bootstrap, SweetAlert2 y API web

Finalidad

Manual práctico para comprobar lo que realmente ve y hace una persona en una aplicación ASP.NET Core: navegación, formularios, validación, tablas, diálogos, temas y diseño responsive.

Al terminar este manual podrás:

- Instalar Playwright con xUnit v3 y los navegadores necesarios.
- Localizar elementos por rol, etiqueta, texto y test id sin selectores frágiles.
- Aprovechar auto-waiting y aserciones reintentables; eliminar esperas fijas.
- Probar formularios Razor, validación, SweetAlert2, temas y vistas responsive.
- Aislar datos y estado del navegador entre pruebas.
- Diagnosticar fallos con trazas, capturas, vídeo y ejecución visible.
- Ejecutar la suite en CI sobre Chromium y, cuando aporte valor, Firefox y WebKit.

Edición didáctica 2026 · .NET 10

Índice

1. Qué prueba Playwright y qué no	4
2. Instalación con xUnit v3	5
Primera prueba	5
3. Arrancar la aplicación bajo prueba	6
Opción sencilla: proceso externo	6
Fixture que controla el proceso	6
Puerto	7
4. Localizadores resistentes	8
Orden de preferencia	8
Strict mode	8
5. Auto-waiting y aserciones web	9
Lo que debe evitarse	9
6. Navegación y formularios Razor Pages	10
Crear un registro	10
Validación	10
Archivos	10
7. Tablas, filtros, ordenación y paginación	12
Filtrar con debounce	12
Una fila concreta	12
Paginación	12
8. SweetAlert2, modales y diálogos	13
Diálogo nativo	13
9. Temas, localStorage y Bootswatch	15

Preparar localStorage antes de navegar	15
10. Responsive, móvil y accesibilidad	16
Viewport	16
Navbar móvil	16
Comprobaciones útiles	16
11. Peticiones, rutas y API desde Playwright	17
Esperar una respuesta	17
Interceptar una API externa	17
APIRequestContext	17
12. Autenticación y estado guardado	18
13. Aislamiento, datos y paralelismo	19
Estrategias	19
Paralelismo	19
14. Diagnóstico: trazas, capturas y ejecución visible	20
Captura	20
Trace Viewer	20
Modo visible y depuración	20
15. Ejecución en varios navegadores y CI	21
Configuración por variables	21
Pipeline	21
16. Errores frecuentes	22
Orden de investigación	22
17. Suite modelo para una aplicación Razor Pages	23
Actividades propuestas	23
18. Referencias	24

1. Qué prueba Playwright y qué no

Playwright controla navegadores reales mediante una API. Comprueba el sistema renderizado: HTML, CSS, JavaScript, peticiones y comportamiento del navegador. Es la capa adecuada para flujos críticos, pero no reemplaza miles de pruebas unitarias rápidas.

Pregunta	Herramienta más directa
¿La regla suma un punto?	xUnit unitario
¿GET /api/preguntas devuelve JSON?	WebApplicationFactory
¿El botón abre la confirmación y borra?	Playwright
¿La navbar se pliega en móvil?	Playwright con viewport
¿La consulta LINQ traduce y pagina?	integración EF Core

Flujo crítico

Reserva pruebas de navegador para operaciones que causarían un problema real si fallasen: inicio de sesión, alta, compra, edición, borrado y navegación principal. Cubrir cada combinación visual vuelve la suite lenta y costosa.

2. Instalación con xUnit v3

BASH

```
1 dotnet new xunit -n TrivialApi.Playwright
2 dotnet add TrivialApi.Playwright package Microsoft.Playwright.Xunit.v3
3 dotnet build TrivialApi.Playwright
4
5 # Windows PowerShell
6 pwsh TrivialApi.Playwright/bin/Debug/net10.0/playwright.ps1 install
7
8 # Linux/macOS
9 pwsh TrivialApi.Playwright/bin/Debug/net10.0/playwright.ps1 install --with-deps
```

La ruta del script depende de la configuración y del framework. Compila antes para generarlo. En Windows también puede ejecutarse con powershell si pwsh no está instalado. En CI, --with-deps instala dependencias del sistema cuando la imagen lo permite.

Primera prueba

CSHARP

```
1 using Microsoft.Playwright;
2 using Microsoft.Playwright.Xunit.v3;
3
4 public sealed class InicioTests : PageTest
5 {
6     [Fact]
7     public async Task Inicio_MuestraTituloPrincipal()
8     {
9         await Page.GotoAsync("http://localhost:5000/");
10
11         await Expect(Page.GetByRole(AriaRole.Heading,
12             new() { Name = "Trivial" })))
13             .ToBeVisibleAsync();
14     }
15 }
```

PageTest proporciona una página nueva y aislada para cada test mediante un BrowserContext. Playwright reutiliza el proceso del navegador para ser eficiente, pero las cookies y localStorage no se comparten entre pruebas salvo que se configure expresamente.

3. Arrancar la aplicación bajo prueba

Opción sencilla: proceso externo

1. Arranca la aplicación en un terminal con `dotnet run --urls http://127.0.0.1:5080`.
2. Espera a que el log indique que escucha.
3. Ejecuta `dotnet test` en otro terminal.
4. Detén la aplicación al terminar.

Fixture que controla el proceso

CSHARP

```
1 public sealed class AplicacionFixture : IAsyncLifetime
2 {
3     private Process? _proceso;
4     public string BaseUrl { get; } = "http://127.0.0.1:5080";
5
6     public async ValueTask InitializeAsync()
7     {
8         _proceso = Process.Start(new ProcessStartInfo
9         {
10             FileName = "dotnet",
11             Arguments = "run --no-build --urls " + BaseUrl,
12             WorkingDirectory = "../TrivialApi",
13             RedirectStandardOutput = true,
14             RedirectStandardError = true,
15             UseShellExecute = false
16         });
17
18         await EsperarDisponibleAsync($"{BaseUrl}/health");
19     }
20
21     public ValueTask DisposeAsync()
22     {
23         if (_proceso is { HasExited: false }) _proceso.Kill(true);
24         _proceso?.Dispose();
25         return ValueTask.CompletedTask;
26     }
27 }
```

No uses un retraso fijo

`Thread.Sleep(5000)` es lento cuando la aplicación arranca rápido e insuficiente cuando arranca despacio. Consulta repetidamente un endpoint de salud con un timeout total y registra la salida del proceso si no llega a estar disponible.

Puerto

Un puerto fijo simplifica clase, pero falla si ya está ocupado o si varias suites corren a la vez. En automatización asigna un puerto libre, pásalo a la aplicación y comunica la BaseUrl a las pruebas. Usa 127.0.0.1 para evitar diferencias de resolución de localhost.

4. Localizadores resistentes

Orden de preferencia

1. Rol y nombre accesible: `GetByRole`.
2. Etiqueta de formulario: `GetByLabel`.
3. Placeholder cuando sea parte real de la UX.
4. Texto visible estable: `GetByText`.
5. Atributo `data-testid` para elementos sin semántica adecuada.
6. CSS o XPath solo como último recurso.

CSHARP

```
1  await Page.GetByRole(AriaRole.Link,  
2      new() { Name = "Preguntas" }).ClickAsync();  
3  
4  await Page.GetByLabel("Enunciado").FillAsync(  
5      "¿Cuál es la capital de Francia?");  
6  
7  await Page.GetByRole(AriaRole.Button,  
8      new() { Name = "Guardar" }).ClickAsync();  
9  
10 Locator fila = Page.GetByRole(AriaRole.Row)  
11     .Filter(new() { HasText = "capital de Francia" });
```

Strict mode

Una acción sobre un locator debe identificar un único elemento. Si hay dos botones Guardar, Playwright falla para revelar la ambigüedad. Reduce el ámbito con una fila, diálogo o formulario; no selecciones el primero sin entender por qué hay varios.

CSHARP

```
1  Locator dialogo = Page.GetByRole(AriaRole.Dialog);  
2  await dialogo.GetByRole(AriaRole.Button,  
3      new() { Name = "Eliminar" }).ClickAsync();
```

5. Auto-waiting y aserciones web

Antes de hacer clic, Playwright espera que el elemento exista, sea visible, estable, reciba eventos y esté habilitado. Las aserciones Expect vuelven a consultar hasta que se cumplen o vence el timeout.

```
CSHARP
1  await Expect(Page).ToHaveURLAsync(
2      new Regex("/Preguntas$"));
3
4  await Expect(Page.GetByRole(AriaRole.Alert))
5      .ToContainTextAsync("guardado");
6
7  await Expect(Page.GetByRole(AriaRole.Row))
8      .ToHaveCountAsync(11);
9
10 await Expect(Page.GetByText("Cargando"))
11     .ToBeHiddenAsync();
```

Lo que debe evitarse

```
CSHARP
1  // Frágil y lento: no hacerlo.
2  await Task.Delay(3000);
3  string texto = await Page.TextContentAsync("body");
4  Assert.Contains("guardado", texto);
```

La espera fija supone un tiempo que quizá no sea suficiente. La lectura manual del body pierde la semántica y no reintenta. Usa la aserción de Playwright más específica.

Timeout

Aumentar todos los timeout oculta lentitud y no arregla una condición incorrecta. Ajusta solo operaciones justificadamente largas y conserva trazas para saber dónde se agotó la espera.

6. Navegación y formularios Razor Pages

Crear un registro

CSHARP

```
1 [Fact]
2 public async Task CrearPregunta_DatosValidos_ApareceEnListado()
3 {
4     await Page.GotoAsync($"{BaseUrl}/Preguntas/Crear");
5
6     await Page.GetByLabel("Enunciado")
7         .FillAsync("¿Qué planeta es conocido como el planeta rojo?");
8     await Page.GetByLabel("Categoría").SelectOptionAsync(
9         new SelectOptionValue { Label = "Ciencia" });
10    await Page.GetByLabel("Respuesta correcta").FillAsync("Marte");
11    await Page.GetByRole(AriaRole.Button,
12        new() { Name = "Guardar" }).ClickAsync();
13
14    await Expect(Page).ToHaveURLAsync(new Regex("/Preguntas$"));
15    await Expect(Page.GetByText("planeta rojo"))
16        .ToBeVisibleAsync();
17 }
```

Validación

CSHARP

```
1 [Fact]
2 public async Task CrearPregunta_SinTexto_MuestraError()
3 {
4     await Page.GotoAsync($"{BaseUrl}/Preguntas/Crear");
5     await Page.GetByRole(AriaRole.Button,
6         new() { Name = "Guardar" }).ClickAsync();
7
8     await Expect(Page.GetByText("Escribe el enunciado.")).
9         .ToBeVisibleAsync();
10    await Expect(Page).ToHaveURLAsync(
11        new Regex("/Preguntas/Crear"));
12 }
```

Decide si quieres verificar la validación de cliente o servidor. Si scripts de validación están activos, el navegador puede impedir el POST. Para una prueba específica del servidor, puede enviarse una petición HTTP o desactivar la validación del formulario de forma controlada.

Archivos

CSHARP

```
1 await Page.GetByLabel("Imagen").SetInputFilesAsync(  
2     Path.Combine("Recursos", "imagen-prueba.png"));
```

7. Tablas, filtros, ordenación y paginación

Filtrar con debounce

CSHARP

```
1  await Page.GotoAsync($"{BaseUrl}/Preguntas");
2  await Page.GetByLabel("Buscar").FillAsync("capital");
3
4  await Expect(Page).ToHaveURLAsync(
5      new Regex("buscar=capital"));
6
7  Locator filas = Page.GetByRole(AriaRole.Row);
8  await Expect(filas.Filter(new() { HasText = "capital" })))
9      .Not.ToHaveCountAsync(0);
```

No añadas `Task.Delay(500)` para copiar el debounce. Espera el efecto observable: cambio de URL, respuesta concreta o actualización del listado.

Una fila concreta

CSHARP

```
1  Locator fila = Page.GetByRole(AriaRole.Row)
2      .Filter(new() { HasText = "¿Capital de Francia?" });
3
4  await Expect(fila).ToContainTextAsync("Geografía");
5  await fila.GetByRole(AriaRole.Link,
6      new() { Name = "Editar" }).ClickAsync();
```

Paginación

- Comprueba el enlace por su nombre accesible, no por `.pagination li:nth-child`.
- Verifica que filtros y orden se mantienen en la URL.
- Comprueba desactivación de Anterior en la primera página y Siguiente en la última.
- No presupongas el número total si otros tests pueden modificar los datos.

8. SweetAlert2, modales y diálogos

SweetAlert2 construye un diálogo dentro del DOM, por lo que se localiza por `role=dialog` y por sus botones. No debe confundirse con los diálogos nativos `alert/confirm/prompt`, que se gestionan mediante el evento `Dialog`.

CSHARP

```
1 [Fact]
2 public async Task Eliminar_Cancelar_ConservaRegistro()
3 {
4     await Page.GotoAsync($"{BaseUrl}/Categorias");
5     Locator fila = Page.GetByRole(AriaRole.Row)
6         .Filter(new() { HasText = "Historia" });
7
8     await fila.GetByRole(AriaRole.Button,
9         new() { Name = "Eliminar" }).ClickAsync();
10
11     Locator dialogo = Page.GetByRole(AriaRole.Dialog);
12     await Expect(dialogo).ToBeVisibleAsync();
13     await dialogo.GetByRole(AriaRole.Button,
14         new() { Name = "Cancelar" }).ClickAsync();
15
16     await Expect(fila).ToBeVisibleAsync();
17 }
```

CSHARP

```
1 [Fact]
2 public async Task Eliminar_Confirmar_EliminaRegistro()
3 {
4     // ... localizar fila y pulsar Eliminar ...
5     await Page.GetByRole(AriaRole.Dialog)
6         .GetByRole(AriaRole.Button,
7             new() { Name = "Eliminar" })
8         .ClickAsync();
9
10     await Expect(Page.GetByText("Registro eliminado"))
11         .ToBeVisibleAsync();
12     await Expect(Page.GetByText("Historia"))
13         .ToBeHiddenAsync();
14 }
```

Diálogo nativo

CSHARP

```
1 Page.Dialog += async (_, dialogo) =>
2 {
3     Assert.Equal(DialogType.Confirm, dialogo.Type);
4     await dialogo.AcceptAsync();
5 };
6
7 await Page.GetByRole(AriaRole.Button,
8     new() { Name = "Eliminar" }).ClickAsync();
```

9. Temas, localStorage y Bootswatch

CSHARP

```
1 [Fact]
2 public async Task TemaOscuro_SeMantieneTrasRecargar()
3 {
4     await Page.GotoAsync(BaseUrl);
5     await Page.GetByRole(AriaRole.Button,
6         new() { Name = "Seleccionar tema" }).ClickAsync();
7     await Page.GetByRole(AriaRole.Menuitem,
8         new() { Name = "Darkly" }).ClickAsync();
9
10    await Expect(Page.Locator("html"))
11        .ToHaveAttributeAsync("data-bs-theme", "dark");
12
13    Assert.Equal("bootswatch-darkly",
14        await Page.EvaluateAsync<string>(
15            "() => localStorage.getItem('temaTrivial')"));
16
17    await Page.ReloadAsync();
18    await Expect(Page.Locator("#temaCss"))
19        .ToHaveAttributeAsync("href", new Regex("darkly"));
20 }
```

Preparar localStorage antes de navegar

CSHARP

```
1 await Page.AddInitScriptAsync("""
2     localStorage.setItem('temaTrivial', 'bootstrap-dark');
3     """);
4
5 await Page.GotoAsync(BaseUrl);
```

AddInitScript se ejecuta antes del script de la página y evita el destello del tema inicial. No compares colores RGB exactos salvo que formen parte del contrato; es más estable comprobar el atributo, la hoja activa y la legibilidad de elementos críticos.

10. Responsive, móvil y accesibilidad

Viewport

CSHARP

```
1 public override BrowserNewContextOptions ContextOptions() => new()
2 {
3     ViewportSize = new ViewportSize
4     {
5         Width = 390,
6         Height = 844
7     },
8     IsMobile = true,
9     HasTouch = true
10 };
```

Navbar móvil

CSHARP

```
1 await Page.GotoAsync(BaseUrl);
2
3 Locator botonMenu = Page.GetByRole(AriaRole.Button,
4     new() { Name = "Alternar navegación" });
5 await Expect(botonMenu).ToBeVisibleAsync();
6
7 await botonMenu.ClickAsync();
8 await Expect(Page.GetByRole(AriaRole.Link,
9     new() { Name = "Preguntas" })))
10 .ToBeVisibleAsync();
```

Comprobaciones útiles

- No existe desplazamiento horizontal inesperado.
- La tabla dispone de contenedor responsive o alternativa en tarjetas.
- Los botones táctiles tienen tamaño suficiente y no se solapan.
- El foco puede recorrer la página en orden lógico.
- Los botones de icono tienen nombre accesible.
- Los diálogos atrapan y devuelven el foco correctamente.

CSHARP

```
1 int anchoDocumento = await Page.EvaluateAsync<int>(
2     "() => document.documentElement.scrollWidth");
3 int anchoVista = await Page.EvaluateAsync<int>(
4     "() => document.documentElement.clientWidth");
5
6 Assert.True(anchoDocumento <= anchoVista + 1);
```

11. Peticiones, rutas y API desde Playwright

Esperar una respuesta

CSHARP

```
1 var respuesta = await Page.RunAndWaitForResponseAsync(
2     async () => await Page.GetByLabel("Buscar").FillAsync("nasa"),
3     response => response.Url.Contains("/api/") && response.Status == 200);
4
5 Assert.Equal(200, respuesta.Status);
```

Interceptar una API externa

CSHARP

```
1 await Page.RouteAsync("**/api/character/**", async route =>
2 {
3     await route.FulfillAsync(new RouteFulfillOptions
4     {
5         Status = 200,
6         ContentType = "application/json",
7         Body = """
8         { "id": 1, "name": "Rick", "status": "Alive" }
9         """
10    });
11 });
```

El mock de red hace determinista la interfaz que consume terceros y permite probar 404, 500, timeout y JSON inválido. No lo uses para sustituir todas las pruebas de integración de tu propia API: alguna prueba debe atravesar el backend real.

APIRequestContext

Playwright también puede enviar peticiones HTTP sin página. Resulta útil para crear datos de preparación o limpiar el escenario, pero las pruebas puramente de API suelen expresarse mejor con `WebApplicationFactory` y `xUnit`.

12. Autenticación y estado guardado

Iniciar sesión por interfaz en cada test es lento. Comprueba el login visual en una prueba; para el resto, autentica una vez, guarda `StorageState` y crea contextos desde ese estado. Nunca subas el archivo si contiene cookies o tokens reales.

CSHARP

```
1  await Page.GotoAsync($"{BaseUrl}/Identity/Account/Login");
2  await Page.GetByLabel("Correo electrónico").FillAsync("profesor@ejemplo.es");
3  await Page.GetByLabel("Contraseña").FillAsync("ClaveTemporal123!");
4  await Page.GetByRole(AriaRole.Button,
5      new() { Name = "Iniciar sesión" }).ClickAsync();
6
7  await Page.Context.StorageStateAsync(new()
8  {
9      Path = "playwright/.auth/profesor.json"
10 });
```

Datos de prueba

Crea usuarios y contraseñas exclusivos del entorno Testing. El estado de autenticación es sensible incluso si pertenece a una cuenta de pruebas y debe ignorarse en control de versiones.

13. Aislamiento, datos y paralelismo

Estrategias

Estrategia	Ventaja	Coste
Base nueva por test	aislamiento máximo	más tiempo
Transacción por test	rápida	difícil si navegador y servidor usan conexiones distintas
Datos con identificador único	paralelizable	requiere limpieza
Reset de base entre tests	estado conocido	operación de infraestructura

CSHARP

```
1 string sufijo = Guid.NewGuid().ToString("N")[..8];  
2 string nombre = $"Categoria-{sufijo}";
```

Una prueba no debe borrar por texto genérico ni asumir que ocupa la primera fila. Guarda el identificador o usa un nombre único. Si falla antes de limpiar, el siguiente test no debe quedar roto.

Paralelismo

PageTest aísla el navegador, no la base de datos. Si la aplicación usa una base compartida, limita el paralelismo de esa colección o diseña datos independientes. Ejecuta la suite varias veces y en orden aleatorio cuando el corredor lo permita para descubrir acoplamientos.

14. Diagnóstico: trazas, capturas y ejecución visible

Captura

CSHARP

```
1  await Page.ScreenshotAsync(new()  
2  {  
3      Path = "artifacts/fallo.png",  
4      FullPage = true  
5  });
```

Trace Viewer

Una traza conserva acciones, DOM, red, consola y capturas alrededor de cada paso. Es la mejor evidencia de un fallo en CI. Configura su retención en primer reintento o al fallar para no generar artefactos enormes en cada test.

BASH

```
1  pwsh bin/Debug/net10.0/playwright.ps1 show-trace artifacts/trace.zip
```

Modo visible y depuración

BASH

```
1  $env:HEADED="1"  
2  $env:PWDEBUG="1"  
3  dotnet test --filter "Name~Eliminar_Confirmar"
```

- Conserva consola del navegador y peticiones fallidas.
- Incluye URL actual y nombre del test en artefactos.
- No hagas capturas antes de que aparezca el estado que interesa.
- Evita ocultar un fallo con reintentos ilimitados: un flaky sigue siendo un defecto.

15. Ejecución en varios navegadores y CI

Chromium suele ser el navegador principal por velocidad y herramientas. Añade Firefox y WebKit para detectar incompatibilidades cuando el proyecto lo requiera; no multipliques toda la matriz si no aporta información.

Configuración por variables

CSHARP

```
1 public override BrowserNewContextOptions ContextOptions()
2 {
3     return new()
4     {
5         BaseUrl = Environment.GetEnvironmentVariable("BASE_URL")
6             ?? "http://127.0.0.1:5080",
7         Locale = "es-ES",
8         TimezoneId = "Europe/Madrid",
9         ColorScheme = ColorScheme.Light
10    };
11 }
```

Pipeline

1. Restaurar y compilar la aplicación y las pruebas.
2. Instalar navegadores y dependencias usando el script generado.
3. Arrancar la aplicación con entorno Testing y base aislada.
4. Esperar el endpoint de salud.
5. Ejecutar dotnet test con resultados y trazas al fallar.
6. Publicar artefactos y detener el proceso aunque la suite falle.

Entorno reproducible

Fija zona horaria, locale, viewport y datos. Una fecha formateada o un separador decimal puede variar entre el ordenador del alumno y Linux en CI.

16. Errores frecuentes

Síntoma	Causa y solución
Executable doesn't exist	no se instalaron navegadores tras compilar; ejecutar playwright.ps1 install
ERR_CONNECTION_REFUSED	aplicación no arrancada, URL/puerto erróneo o fallo durante inicio
Strict mode violation	locator coincide con varios; acotar por región, fila o diálogo
Timeout al pulsar	elemento cubierto, deshabilitado o selector incorrecto; mirar traza
Pasa local, falla CI	locale, tiempo, ruta, mayúsculas, dependencia o carrera
SweetAlert no encontrado	es diálogo DOM; usar GetByRole(Dialog), no evento Dialog
Datos duplicados	tests comparten base; usar id único y limpieza
Cursor vuelve al inicio	página recargada por filtro; restaurar foco o actualizar parcialmente

Orden de investigación

1. Leer el mensaje y la acción exacta que agotó el timeout.
2. Abrir la traza y comprobar DOM, URL y red en ese instante.
3. Reproducir solo el test en modo visible.
4. Revisar logs del servidor y consola del navegador.
5. Eliminar cualquier Task.Delay añadido como parche y esperar un estado observable.

17. Suite modelo para una aplicación Razor Pages

Área	Casos de navegador
Navegación	inicio, enlaces principales, pie, ayuda API y 404
Categorías	crear, validación repetida, editar, cancelar/confirmar borrado
Preguntas	crear, filtrar, paginar, editar y límite de borrado
Cliente Trivial	cargar preguntas, responder, mostrar corrección y volver a API
Temas	selector completo, persistencia, claro/oscuro y carrusel visible
Responsive	navbar, tabla/tarjetas, ausencia de overflow y botones accesibles
Errores	API caída, lista vacía y mensaje recuperable

Actividades propuestas

1. Escribe una prueba que filtre mientras escribe y espere el cambio de URL sin retardos fijos.
2. Prueba cancelar y confirmar el mismo borrado SweetAlert2 usando una fila única.
3. Intercepta una API externa para devolver 500 y verifica el mensaje de error de la página.
4. Ejecuta la navbar a 390 px y comprueba que todos los enlaces son accesibles tras abrir el menú.
5. Provoca un fallo, guarda una traza y documenta cómo localizar la petición responsable.

18. Referencias

[Instalación de Playwright para .NET](#)

[Escribir pruebas](#)

[Corredores xUnit y xUnit v3](#)

[Aserciones web](#)

[Ejecución y filtros](#)

[Generador de pruebas](#)

[Trace Viewer](#)

[Capturas de pantalla](#)